

setup

overhead

tag

```
In[ ]:= home = "projects/bae/hyperion/simulation/confidence/";
Get["utility_modules.m", Path → dirPack];
Needs["DatabaseLink`"];
stamp1;
```

... CreateDirectory: /home/dantopa/Mathematica_files/io/ already exists.

... CreateDirectory: /mnt/T7-Shield/Dropbox/_mm/io/projects/ already exists.

... CreateDirectory: /mnt/T7-Shield/Dropbox/_mm/io/projects/bae/ already exists.

... General: Further output of CreateDirectory::eexist will be suppressed during this calculation. ⓘ

maximum memory: 0.222743 GB

seed file: /home/dantopa/Mathematica_files/nb/seed_23_01.nb

user: dantopa, CPU: isomer, MM v. 15.0.1 for Linux x86

date: Aug 27, 2026, time: 00:11:49

nb: /home/dantopa/Mathematica_files/nb/projects/bae/hyperion/simulation/confidence/
database-04.nb

modules, functions, settings, ...

database connections

```

In[ ]:= (* open db connection *)
dbFile = dirData <> "confidence.sqlite";

(* clear the board *)
If[FileExistsQ[dbFile], DeleteFile[dbFile]];

(*open fresh database*)
db = OpenSQLConnection[JDBC["SQLite", dbFile]];

(* create table *)
SQLExecute[db, "
CREATE TABLE confidence (
  form_number    INTEGER NOT NULL,
  run_uuid       TEXT     NOT NULL,
  wbs_primitive  TEXT     NOT NULL,
  overall        INTEGER NOT NULL CHECK (overall IN (-1,0,1)),
  assess_a       INTEGER NOT NULL CHECK (assess_a IN (-1,0,1)),
  assess_b       INTEGER NOT NULL CHECK (assess_b IN (-1,0,1)),
  assess_c       INTEGER NOT NULL CHECK (assess_c IN (-1,0,1))
)
"];

(* insertion statement *)
insertSQL = "
INSERT INTO confidence
  (form_number, run_uuid, wbs_primitive,
   overall, assess_a, assess_b, assess_c)
VALUES (?, ?, ?, ?, ?, ?, ?)
";

```

random numbers

```

In[ ]:= SeedRandom[1];

In[ ]:= (* create score types *)

```

```

In[ ]:= Clear[ScoreNeutral, ScoreGreat, ScoreGood];
ScoreNeutral[] := Module[{c, x},
  x = RandomVariate[UniformDistribution[{-1, 1}]];
  If[x < - $\frac{2}{3}$ , c = -1; Return[c]];
  If[x <  $\frac{2}{3}$ , c = 0; Return[c]];
  c = 1;
  Return[c]]

In[ ]:= ScoreGreat[] := Module[{c, x},
  x = RandomVariate[UniformDistribution[{-1, 1}]];
  If[x < - $\frac{7}{8}$ , c = -1; Return[c]];
  If[x < - $\frac{3}{4}$ , c = 0; Return[c]];
  c = 1;
  Return[c]]

In[ ]:= ScoreGood[] := Module[{c, x},
  x = RandomVariate[UniformDistribution[{-1, 1}]];
  If[x < - $\frac{3}{4}$ , c = -1; Return[c]];
  If[x < 0, c = 0; Return[c]];
  c = 1;
  Return[c]]

In[ ]:= (* aggregate scores *)

In[ ]:= cstarGreat[] := {ScoreGreat[], ScoreGreat[], ScoreGreat[]}
cstarGood[] := {ScoreGood[], ScoreGood[], ScoreGood[]}
cstarNeutral[] := {ScoreNeutral[], ScoreNeutral[], ScoreNeutral[]}
cstarBad[] := -cstarGood[]
cstarAwful[] := -cstarGreat[]

In[ ]:= (* population counts *)

In[ ]:= HowMany[] := Round[RandomVariate[UniformDistribution[{0.15, 1}]] 100];

```

generate populations

primitives

```

In[ ]:= (* construct *)

```

```

In[ ]:= wbs1k = Table[StringJoin["1.", ToString[k]], {k, 3, 8}]
Out[ ]:= {1.3, 1.4, 1.5, 1.6, 1.7, 1.8}

In[ ]:= wbs12k = Table[StringJoin["1.2.", ToString[k]], {k, 4}]
Out[ ]:= {1.2.1, 1.2.2, 1.2.3, 1.2.4}

In[ ]:= wbs11k = Table[StringJoin["1.1.", ToString[k]], {k, 2, 7}]
Out[ ]:= {1.1.2, 1.1.3, 1.1.4, 1.1.5, 1.1.6, 1.1.7}

In[ ]:= wbs111k = Table[StringJoin["1.1.1.", ToString[k]], {k, 3}]
Out[ ]:= {1.1.1.1, 1.1.1.2, 1.1.1.3}

In[ ]:= (* assign *)

In[ ]:= remainderList = Join[wbs1k, wbs11k, wbs12k, wbs111k]
Out[ ]:= {1.3, 1.4, 1.5, 1.6, 1.7, 1.8, 1.1.2, 1.1.3, 1.1.4, 1.1.5,
          1.1.6, 1.1.7, 1.2.1, 1.2.2, 1.2.3, 1.2.4, 1.1.1.1, 1.1.1.2, 1.1.1.3}

```

populate db

```

In[ ]:= SeedRandom[1];

```

wbs111k

```

In[ ]:= uuid = CreateUUID[];
nForms = 0;
nLocal = HowMany[];
complement = Table[{0, 0, 0, 0}, {nLocal}];

In[ ]:= (* great: best scores *)
thisList = wbs111k[[1]];
Print[nLocal, ": ", thisList];
Do[
  (* generate confidence scores *)
  lst = Flatten[Join[{ScoreGreat[], cstarGreat[]}]];
  (* store the complement of this rating *)
  complement[[j]] = -lst;
  record = {++nForms, uuid, thisList, Sequence @@ lst};
  SQLExecute[db, insertSQL, record];
  , {j, nLocal}];
84: 1.1.1.1

```

```

In[ ]:= (* awful: worst scores; exactly cancels great scores *)
thisList = wbs11k[[2]];
Print[nLocal, ": ", thisList];
Do[
  record = {++nForms, uuid, thisList, Sequence @@ complement[[j]]};
  SQLExecute[db, insertSQL, record];
  , {j, nLocal}];
84: 1.1.1.2

```

```

In[ ]:= (* typical *)
nLocal = HowMany[];
thisList = wbs11k[[3]];
Print[nLocal, ": ", thisList];
lst = Flatten[Join[{ScoreNeutral[], cstarNeutral[]}]];
Do[
  record = {++nForms, uuid, thisList, Sequence @@ lst};
  SQLExecute[db, insertSQL, record];
  , {j, nLocal}];
94: 1.1.1.3

```

wbs12k

```

In[ ]:= (* good *)
thisList = wbs12k[[1]];
nLocal = HowMany[];
Print[nLocal, ": ", thisList];
complement = Table[{0, 0, 0, 0}, {nLocal}];
Do[
  (* generate confidence scores *)
  lst = Flatten[Join[{ScoreGood[], cstarGood[]}]];
  (* store the complement of this rating *)
  complement[[j]] = -lst;
  record = {++nForms, uuid, thisList, Sequence @@ lst};
  SQLExecute[db, insertSQL, record];
  , {j, nLocal}];
18: 1.2.1

```

```

In[ ]:= (* bad *)
thisList = wbs1k[[2]];
Print[nLocal, ": ", thisList];
Do[
  record = {++nForms, uuid, thisList, Sequence @@ complement[[j]]};
  SQLExecute[db, insertSQL, record];
  , {j, nLocal}];
18: 1.4

```

```

In[ ]:= (* good *)
thisList = wbs12k[[3]];
nLocal = HowMany[];
Print[nLocal, ": ", thisList];
Do[
  (* generate confidence scores *)
  lst = Flatten[Join[{ScoreGood[], cstarGood[]}]];
  (* store the complement of this rating *)
  record = {++nForms, uuid, thisList, Sequence @@ lst};
  SQLExecute[db, insertSQL, record];
  , {j, nLocal}];
29: 1.2.3

```

```

In[ ]:= (* bad *)
thisList = wbs12k[[4]];
nLocal = HowMany[];
Print[nLocal, ": ", thisList];
Do[
  record = {++nForms, uuid, thisList, Sequence @@ lst};
  SQLExecute[db, insertSQL, record];
  , {j, nLocal}];
85: 1.2.4

```

wbs11k

```

In[ ]:= (* good *)
Do[
  thisList = wbs11k[[1]];
  nLocal = HowMany[];
  Print[nLocal, ": ", thisList];
  Do[
    (* generate confidence scores *)
    lst = Flatten[Join[{ScoreGood[], cstarGood[]}]];
    record = {++nForms, uuid, thisList, Sequence @@ lst};
    SQLExecute[db, insertSQL, record];
    , {j, nLocal}];
  , {l, Length[wbs11k]}];
65: 1.1.2
86: 1.1.3
27: 1.1.4
32: 1.1.5
21: 1.1.6
95: 1.1.7

```

wbs1k: neutral

```
In[*]:= (* good *)
Do[
  thisList = wbs1k[[1]];
  nLocal = HowMany[];
  Print[nLocal, ": ", thisList];
  Do[
    (* generate confidence scores *)
    lst = Flatten[Join[{ScoreNeutral[], cstarNeutral[]}]];
    record = {++nForms, uuid, thisList, Sequence @@ lst};
    SQLExecute[db, insertSQL, record];
    , {j, nLocal}];
  , {1, Length[wbs1k]}];

88: 1.3
96: 1.4
48: 1.5
85: 1.6
16: 1.7
24: 1.8
```

harvest data

```
In[*]:= scores = SQLExecute[db, "
  SELECT overall, assess_a, assess_b, assess_c
  FROM confidence
  ORDER BY form_number
  "];

In[*]:= Last[scores]
Out[*]:= {0, 0, 0, 0}

In[*]:= SQLExecute[db, "SELECT COUNT(*) FROM confidence"]
Out[*]:= {{1095}}

In[*]:= Times @@
  Flatten@{SQLExecute[db, "PRAGMA page_count"], SQLExecute[db, "PRAGMA page_size"]}
Out[*]:= 73 728
```

```
In[ ]:= header = {"form_number", "run_uuid",  
                 "wbs_primitive", "overall", "assess_a", "assess_b", "assess_c"};  
  
data = SQLExecute[db, "SELECT * FROM confidence"];  
  
Export[dirData <> "database.csv", Prepend[data, header], "CSV"]  
Out[ ]:= /home/dantopa/Mathematica_files/io/projects/bae/hyperion/simulation/confidence/data/database  
        .csv  
  
In[ ]:= CloseSQLConnection[db];
```

end